

BaCoN - Tutorial

Thomas Rohde

Feb 22, 2025

This vignette describes how to predict functional buffering between gene pairs using BaCoN, using data from the Cancer Dependency Map (DepMap).

0. Install BaCoN package

```
if(!require(devtools)) {install.packages("devtools")}  
if (F) {devtools::install_github("billmannlab/BaCoN", force = T)}
```

1. Import packages

```
library(BaCoN)  
library(data.table)  
library(stringr)  
library(tidyverse)  
theme_set(theme_light())
```

2. Data preparation

This script depends on DepMap gene expression data as well as Chronos scores, available at the DepMap portal.

Please download the files and place them in the “data” directory.

- Chronos scores: “CRISPRGeneEffect.csv”
- Gene expression: “OmicsExpressionProteinCodingGenesTPMLogp1.csv”

2.1 Import DepMap gene expression and fitness effect data

```
import_depmap <- \(filepath) {  
  object <- as.matrix(data.table::fread(filepath), rownames = 1)  
  colnames(object) <- stringr::str_split_i(colnames(object), " \\(", 1)  
  return(object)  
}  
  
ge_complete <- import_depmap(  
  file.path("../", "data", "OmicsExpressionProteinCodingGenesTPMLogp1.csv"))  
  
chronos_complete <- import_depmap(  
  file.path("../", "data", "CRISPRGeneEffect.csv"))  
  
message(str_c(round(sum(is.na(chronos_complete)) /
```

```

        length(chronos_complete) * 100, 2),
        " percent of values are imputed."))
#> 0.85 percent of values are imputed.

# Impute missing Chronos scores as gene-wise means

for (i in 1:ncol(chronos_complete)) {
  chronos_complete[,i][is.na(chronos_complete[,i])] <- mean(chronos_complete[,i], na.rm = T)
}

```

2.2 Subset expression and fitness effect matrix

The gene expression and the Chronos score matrix are reduced to the cell lines that are represented in both datasets.

```

intersecting_cell_lines <- intersect(rownames(ge_complete), rownames(chronos_complete))

ge_subset <- ge_complete[intersecting_cell_lines,]
chronos_subset <- chronos_complete[intersecting_cell_lines,]

```

The correlations are computed across 1103 intersecting cell lines.

Conceptually, we expect that biologically meaningful buffering predictions require the buffering partner to be expressed in a sufficient number of cell lines and the knockout of the buffered partner to have a certain impact on cellular fitness. Our experience showed that removing genes with low signal by applying a threshold on the expression level improves buffering predictions.

This is only one possible strategy. In contrast to correlation scores, BaCoN scores are compared with and thus influenced by all other gene partners of the respective pair. It is recommended to experiment with different matrix sizes.

To keep runtimes low, we apply very strict thresholds in this vignette. In a comprehensive analysis, it is strongly recommended to cover a larger fraction of the genome, by defining less strict thresholds.

Matrix sizes of ~ 6000-16000 genes on each side of the matrix showed good results in our experiments. The cell lines should be kept at a high number

```

# We restrict the gene space to genes that do not show an expression (log2 TPM+1)
# of > 5 in at least 1000 of the cell lines.

```

```

expression_genes <- names(which(apply(ge_subset >= 5, 2, sum, na.rm = T) >= 1000))

ge_subset <- ge_subset[,expression_genes]

chronos_genes <- names(which(apply(abs(chronos_subset) > 1, 2, sum) >= 30))
chronos_subset <- chronos_subset[,chronos_genes]

```

```

dim(ge_subset)
#> [1] 1103 1460
str(ge_subset)
#> num [1:1103, 1:1460] 7.11 7.15 6.54 6.09 6.53 ...
#> - attr(*, "dimnames")=List of 2
#> ..$ : chr [1:1103] "ACH-001289" "ACH-001538" "ACH-000242" "ACH-000233" ...
#> ..$ : chr [1:1460] "DPM1" "LASP1" "M6PR" "ARF5" ...
dim(chronos_subset)
#> [1] 1103 1768
str(chronos_subset)

```

```
#> num [1:1103, 1:1768] -1.24 -0.869 -0.83 -0.845 -1.109 ...
#> - attr(*, "dimnames")=List of 2
#> ..$ : chr [1:1103] "ACH-001289" "ACH-001538" "ACH-000242" "ACH-000233" ...
#> ..$ : chr [1:1768] "AAMP" "AARS1" "AARS2" "AATF" ...
```

The resulting universe is reduced to 1460 expression genes and 1768 fitness genes.

```
prediction_matrices <- array(
  data = NA,
  dim = c(ncol(ge_subset), ncol(chronos_subset), 4),
  dimnames = list(colnames(ge_subset),
    colnames(chronos_subset),
    c("PCC", "BaCoN", "Cholesky_PCC", "Cholesky_BaCoN")))

str(prediction_matrices)
#> logi [1:1460, 1:1768, 1:4] NA NA NA NA NA NA ...
#> - attr(*, "dimnames")=List of 3
#> ..$ : chr [1:1460] "DPM1" "LASP1" "M6PR" "ARF5" ...
#> ..$ : chr [1:1768] "AAMP" "AARS1" "AARS2" "AATF" ...
#> ..$ : chr [1:4] "PCC" "BaCoN" "Cholesky_PCC" "Cholesky_BaCoN"
```

3.1 Compute PCC correlation matrix:

```
prediction_matrices[,,"PCC"] <- cor(ge_subset, chronos_subset,
  use = "pairwise.complete.obs")
```

3.2 Compute BaCoN matrix:

BaCoN may behave unexpectedly if the input correlation matrix is formatted in an unexpected way. To ensure stable and reliable results, please ensure that the input matrix...

1. is a correlation matrix or composed of comparable values scaling between -1 and 1. BaCoN is currently not designed to work e.g. on negative logarithmic p-values or other association indices.
2. does not contain NAs. This should not cause problems in cases of single NA values, but entire NA rows or columns should be removed from the matrix.
3. should not have duplicated row- or column names. Each gene should be represented in one row and/or column only.

```
prediction_matrices[,,"BaCoN"] <- BaCoN(prediction_matrices[,,"PCC"])
```

4. Cholesky whitening + PCC + BaCoN

Next, we combine BaCoN with the pre-hoc covariance normalization method Cholesky whitening, as proposed in our manuscript. We use the `whitening` package and its `whiten` function.

```
if (F) {install.packages(whitening)}
library(whitening)
#> Lade nötiges Paket: corpcor
```

We whiten both input matrices independently. It is important that the cell lines of both objects are identical and in the “columns” dimensions of the object. Therefore, we transpose the objects before and retranspose them after the whitening.

```
whitened_ge_subset <- t(whiten(t(ge_subset), method = "Cholesky"))
whitened_chronos_subset <- t(whiten(t(chronos_subset), method = "Cholesky"))
```

```
prediction_matrices[,,"Cholesky_PCC"] <- cor(
  whitened_ge_subset,
  whitened_chronos_subset, use = "pairwise.complete.obs")

prediction_matrices[,,"Cholesky_BaCoN"] <- BaCoN(prediction_matrices[,,"Cholesky_PCC"])
```

5. Collect top 100 predictions:

We use the lowest BaCoN score of the top 100 predictions as cutoff. The high density of paralog pairs is clearly visible. This can lead to more than 100 pairs in the prediction set, as the top BaCoN-scored pairs are often tied. We use the PCC score to deal with the tied pairs.

```
predictions <- data.table(data.frame(
  expression_gene = rep(rownames(prediction_matrices),
    ncol(prediction_matrices)),
  chronos_gene = rep(colnames(prediction_matrices),
    each = nrow(prediction_matrices)),
  sapply(dimnames(prediction_matrices)[[3]],
    \(.) as.vector(prediction_matrices[,,.]), simplify = F)))

predictions[order(BaCoN, PCC, decreasing = T), BaCoN_rank := .I]
predictions[order(Cholesky_BaCoN, Cholesky_PCC, decreasing = T), Cholesky_rank := .I]

predictions <- predictions[order(BaCoN, PCC, decreasing = T)]
```

```
str(predictions)
#> Classes 'data.table' and 'data.frame': 2581280 obs. of 8 variables:
#> $ expression_gene: chr "TIMM17B" "TMSB4X" "BAX" "UQCRH" ...
#> $ chronos_gene : chr "TIMM17A" "SOX10" "TP53" "PAX8" ...
#> $ PCC : num 0.403 0.383 0.369 0.368 0.351 ...
#> $ BaCoN : num 0.999 0.999 0.999 0.999 0.999 ...
#> $ Cholesky_PCC : num 0.1531 0.0965 0.0835 0.0919 0.1643 ...
#> $ Cholesky_BaCoN : num 0.999 0.913 0.927 0.678 0.999 ...
#> $ BaCoN_rank : int 1 2 3 4 5 6 7 8 9 10 ...
#> $ Cholesky_rank : int 3 109 81 4198 2 20 68 27 51 23947 ...
#> - attr(*, ".internal.selfref")=<externalptr>
```

```
predictions[1:25, .(expression_gene, chronos_gene, PCC, BaCoN, Cholesky_BaCoN)]
#>   expression_gene chronos_gene      PCC      BaCoN Cholesky_BaCoN
#>   <char>         <char>      <num>      <num>      <num>
#> 1:      TIMM17B      TIMM17A 0.4027963 0.9993804 0.99938042
#> 2:      TMSB4X      SOX10 0.3830977 0.9993804 0.91263941
#> 3:       BAX      TP53 0.3694276 0.9993804 0.92719950
#> 4:      UQCRH      PAX8 0.3676957 0.9993804 0.67781908
#> 5:      PABPC4      PABPC1 0.3513853 0.9993804 0.99938042
#> 6:      CHMP2A      CHMP3 0.3306022 0.9993804 0.97924411
#> 7:       PPA2      PPA1 0.3078310 0.9993804 0.93463445
#> 8:      HNRNPF      HNRNPH1 0.3075358 0.9993804 0.97304833
#> 9:      EIF4A2      EIF4A1 0.3020645 0.9993804 0.94423792
#> 10:      NAPA      NAPG 0.2938160 0.9993804 0.51177200
#> 11:      SARAF      PPP2CA 0.2855697 0.9993804 0.99907063
#> 12:      SLC25A6      DDX3X 0.2788250 0.9993804 0.93649318
#> 13:      NAE1      VPS4B 0.2732776 0.9993804 0.55916976
#> 14:      ARF5      ARF4 0.2618832 0.9993804 0.74008674
```

```

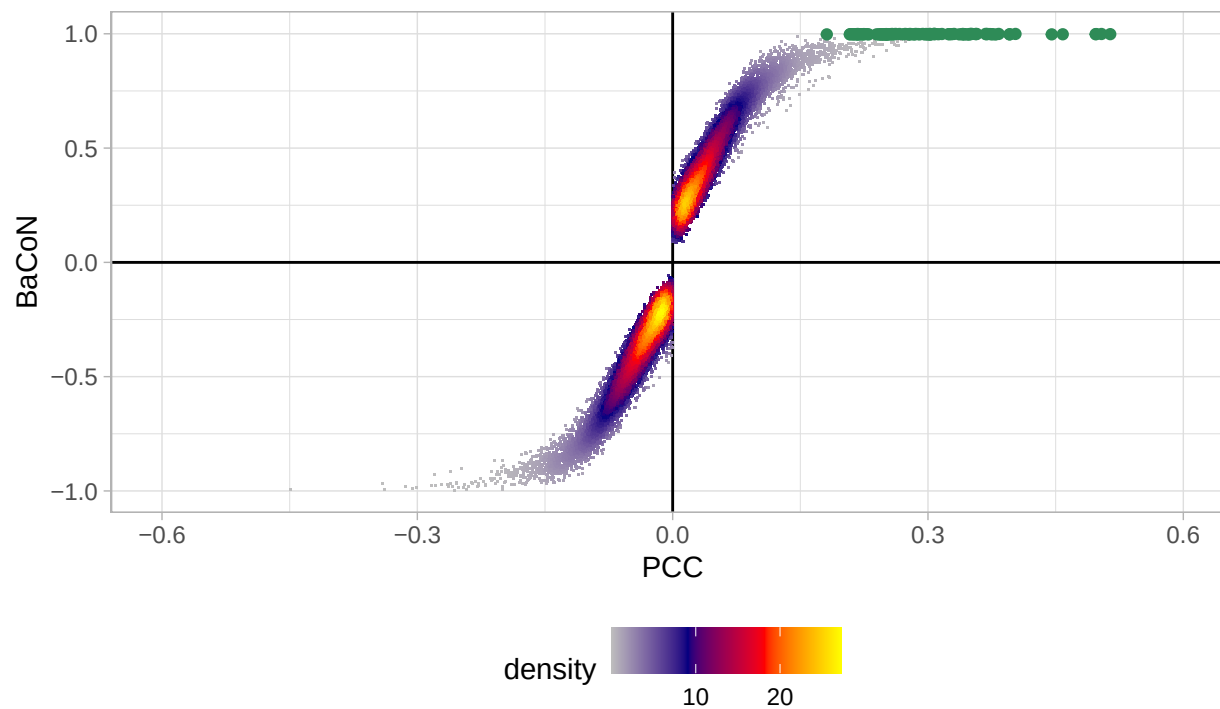
#> 15:      DSTN      CFL1 0.5038338 0.9990706 0.99907063
#> 16:      HMGB2      ILK 0.3572371 0.9990706 0.09665428
#> 17:      CHMP4A      CHMP4B 0.3562702 0.9990706 0.99938042
#> 18:      NARS1      VPS4A 0.3504506 0.9990706 0.96964064
#> 19:      PSME2      THAP1 0.3492945 0.9990706 0.55916976
#> 20:      PPA1      FERMT2 0.3411070 0.9990706 0.67193309
#> 21:      RPL22L1      TP53 0.3163219 0.9990706 -0.06102850
#> 22:      UBE2A      NXT1 0.3064525 0.9990706 0.20260223
#> 23:      PSMD10      NXT1 0.2978793 0.9990706 0.31350682
#> 24:      UQCRH      HNF1B 0.2664924 0.9990706 -0.02881041
#> 25:      RPL35      GLE1 0.2574195 0.9990706 0.44919455
#>      expression_gene chronos_gene      PCC      BaCoN Cholesky_BaCoN

```

6. Show score distribution and predictions

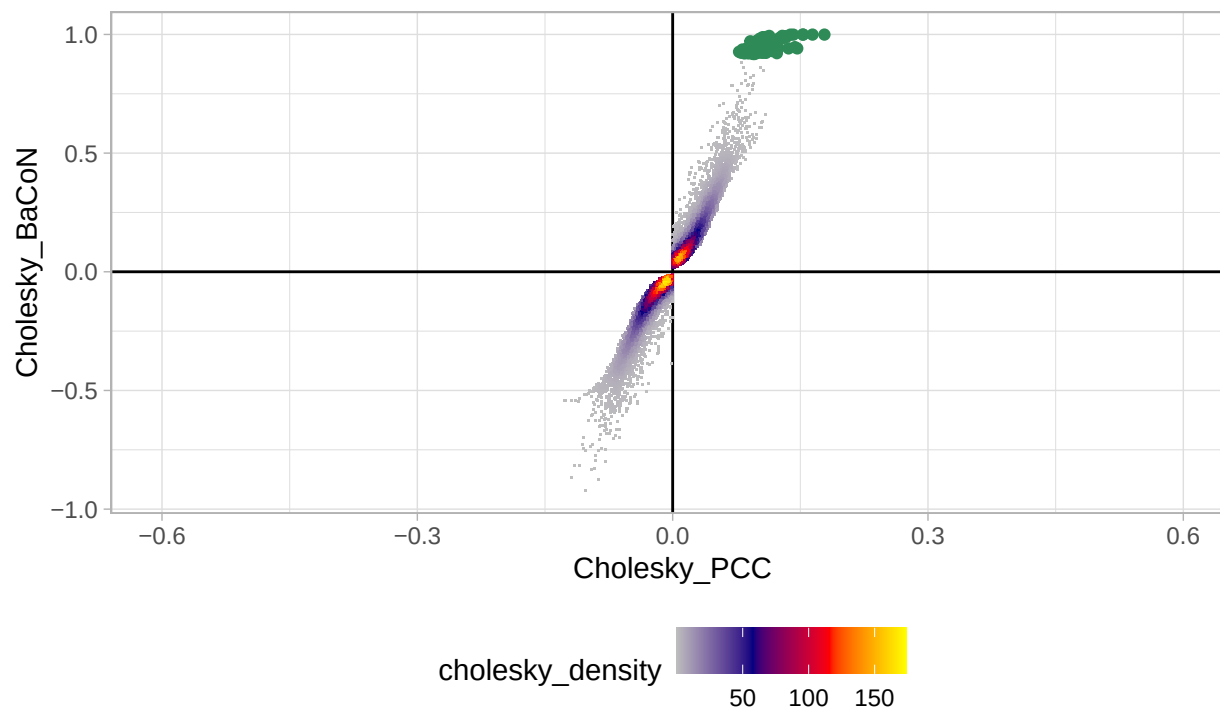
BaCoN vs PCC scores

Heatscatter plots of BaCoN ~ PCC coordinates of gene pairs.
Gene pairs with the top 100 BaCoN scores are colored green.



BaCoN vs PCC scores

Heatscatter plots of Cholesky+BaCoN ~ Cholesky+PCC coordinates of gene pairs.
Gene pairs with the top 100 Cholesky+BaCoN scores are colored green.



Summary

The exponentially increasing runtime of **BaCoN** is due to the greater number of required computations, but also the handling of very large objects. The **data.table** environment is designed to optimize speed and RAM usage when dealing with large datasets. The function is designed to minimize runtime increase caused by data handling, as well as RAM usage.

Future versions of **BaCoN** might offer further runtime improvements.